# Simple Guide: Distributed Food Ordering

Kafka (KRaft) + Spark Batch · Big Data 2026 (UE24CS343AB2) · Team of 4: Member 1 = VM1, Member 2 = VM2, Member 3 = VM3, Member 4 = VM4

| Section | What's in it |
|---|---|
| 1. Big picture | What the assignment does, the 4 machines, the rules |
| 2. Demo day overview | Order of turns, what the evaluator checks, before you start |
| 3. Demo commands | **What to run before the checkpoints, then each checkpoint's commands** (steps 0-22), tagged by member |
| 4-7. Members 1-4 | What each member's part does, and what they show and explain |
| 8-11 | Failure test explained, key words, viva questions, numbers to remember |

## 1. The whole assignment in one minute

A food delivery app has a file of **62 orders** (`order.jsonl`). Some of them are junk: broken records, cancelled orders, bad timestamps, duplicates. We have to:

- **Send** the raw orders through Kafka (VM1 -> VM4).
- **Clean** them and add delivery numbers with Spark (VM4). 62 raw -> **44 good orders**.
- **Send** the cleaned orders back through Kafka (VM4 -> VM1).
- **Summarise** them with Spark (VM1) into two reports: one per restaurant, one per city.
- **Prove** that Kafka keeps working when one broker (VM3) is switched off.

**Analogy.** Kafka is a **post office** that stores letters (messages) in labelled mailboxes (*topics*). It keeps two copies of every letter, one in each branch (VM2 and VM3), so if one branch closes the letters are still there. Spark is the **calculator** each side uses on its own desk after collecting the mail.

### The 4 machines

| Member / VM | Role | What it does in plain words |
|---|---|---|
| Member 1 / VM1 | Producer 1, Consumer 2, Spark Job 2 | Posts the raw orders into Kafka. Later collects the cleaned orders and makes the two final CSV reports. |
| Member 2 / VM2 | Kafka broker 1 + controller | A Kafka server, and also the **boss** of the cluster: it knows which servers are alive and who is in charge of each piece of data. |
| Member 3 / VM3 | Kafka broker 2 | The second Kafka server. Holds a backup copy of everything. This is the one we switch off in the failure test. |
| Member 4 / VM4 | Consumer 1, Spark Job 1, Producer 2 | Collects the raw orders, cleans them and adds metrics with Spark, then posts the cleaned orders back into Kafka. |

### The flow, step by step

```
STAGE 1                                      STAGE 2
VM1  producer1.py                            VM4  producer2.py
      |  topic: orders  (62 msgs)                   |  topic: order_metrics (44 msgs)
      v                                             v
VM4  consumer1.py -> orders_landed.jsonl     VM1  consumer2.py -> order_metrics_landed.jsonl
      v                                             v
VM4  spark_job1.py -> order_metrics.jsonl    VM1  spark_job2.py -> restaurant_summary.csv
      (clean + enrich, 62 -> 44)                                  -> city_summary.csv

Both topics live on the Kafka cluster = VM2 + VM3 (connected over ZeroTier)
Each topic: 2 partitions, replication factor 2
```

## Rules from the spec (the things examiners check)

- Spark never reads Kafka directly. A consumer first saves the data to a local `.jsonl` file; Spark reads that file.
- No HDFS. Everything is on the normal local disk (`outputs/`).
- VM4 must not copy its file to VM1. It must go through Kafka (topic `order_metrics`).
- Both topics: **2 partitions, replication factor 2**, same cluster.
- Failure test: stop **VM3**, never VM2 (VM2 is the only boss).
- Exact duplicates counted once in both Spark jobs. All calculated numbers rounded to 2 decimals. Nothing hardcoded.

# 2. Demo day overview

The evaluator checks **4 checkpoints**. Section 3 has only the commands needed for them: first what to run **before** the checkpoints, then each checkpoint's commands in order. Each step says which member runs it; when you finish, say **"done"** and the member with the next step starts.

## What each member runs

| Member | Before the checkpoints | Checkpoint 1 | Checkpoint 2 | Checkpoint 3 | Checkpoint 4 |
|---|---|---|---|---|---|
| **Member 1** (VM1) | 0 setup, 4 clear outputs | - | 7 Producer 1 | 13 Consumer 2, 14 wc -l, 15 Spark Job 2 | 17 slow producer |
| **Member 2** (VM2) | 0 setup, 1 start broker 1, 3 fresh topics | 5 cluster checks | 6 observer | 11 observer | 16 describe, 19 describe, 21 describe, 22 election |
| **Member 3** (VM3) | 0 setup, 2 start broker 2 | - | - | - | 18 kill broker, 20 restart broker |
| **Member 4** (VM4) | 0 setup, 4 clear outputs | - | 8 Consumer 1, 9 wc -l | 10 Spark Job 1, 12 Producer 2 | - |

## What the evaluator checks

| Checkpoint | Steps | Evidence on screen |
|---|---|---|
| **1.** Brokers reachable, VM2 is the controller, topics exist | 5 | ids 1 and 2; `LeaderId: 1`; `orders` and `order_metrics` |
| **2.** Raw orders flow VM1 -> VM4 | 6-9 | observer prints 3 records; `wc -l` = 62 |
| **3.** Enriched orders flow VM4 -> VM1 | 10-15 | observer prints 3 records; `wc -l` = 44; the two CSVs (accuracy checked on the portal) |
| **4.** Broker failure and recovery | 16-22 | Isr 1,2 -> kill VM3 -> Leader 1 / Isr 1 while the slow producer keeps sending -> restart -> Isr 1,2 -> election puts the leaders back |

**Output accuracy** is graded from this live run only, on the portal `assignment2eval2026.vercel.app`.

## Before you start (everyone)

- `git pull` so every VM has the same code. ZeroTier connected; you can `ping` VM2's ZeroTier IP.
- Use a **Linux / WSL terminal**, not PowerShell. The prompt must look like `user@machine:...$`, not `PS C:\...>`.
- Do step 0 in **every** terminal you'll use.
- Repo scripts are run as `bash kafka/...sh`. If you see `Permission denied` for `./kafka/...`, the copy lost its executable flag: run `chmod +x kafka/*.sh kafka/local/*.sh practice_alone.sh` once, or put `bash` in front.
- Never stop VM2. Don't edit `order.jsonl`.
- Spark's `WARN` lines are normal. Only a `Traceback` means a failure.

# 3. Demo commands, in order

**Demo day:** go down the list. Run only the steps tagged with your member number.

**Practising alone** (one laptop): run **every** step, whatever its tag, using the "Practising alone" box where there is one. Steps 6, 11 and 17 keep running, so do the next step in a second terminal (with step 0 done in it too).

**Copying:** copy one grey box at a time. A line ending in `\` continues on the next line, so copy both lines together. The broker addresses are already filled in: VM2 = `10.31.88.79:9092`, VM3 = `VM3_ZEROTIER_IP:9092`. In the observer commands, replace `teamXX` with your team number.

## One-time setup (the day before, not in front of the TA)

Do this once per machine. It gets the code and creates the broker config files that steps 1, 2 and 20 use, with the right ZeroTier IPs already in them, so the brokers never announce `localhost`.

**Git in one minute.** Our project lives on GitHub: `github.com/Aryan-20-2006/BD-Assignment`. `git clone` downloads the whole project folder once (box A). `git pull` downloads only what has changed since (box A2, and the first lines of step 0). You don't need a GitHub account, and you never need to upload anything: Member 1 does that. Open a terminal on Ubuntu with **Ctrl+Alt+T**. When `sudo` asks for a password, it's your VM's login password, and nothing appears on screen while you type it; that's normal.

### Step A · EVERYONE · Get the code with git, into ~/BD-Assignment (Linux VMs)

```
sudo apt install -y git
cd ~
[ -L ~/BD-Assignment ] && rm ~/BD-Assignment
git clone https://github.com/Aryan-20-2006/BD-Assignment.git ~/BD-Assignment
ls ~/BD-Assignment/kafka
```

**Expect:** the `ls` lists `local topics.sh vm`. Everyone now has the same folder, `~/BD-Assignment`, so every `cd ~/BD-Assignment` in this guide works. The third line only removes an old shortcut with that name, never a real folder. An old zip copy elsewhere can stay; it's no longer used. **Member 4** also creates the Python environment once: `cd ~/BD-Assignment`, `python3 -m venv .venv`, `source .venv/bin/activate`, `pip install -r requirements.txt`. Member 1 (WSL) skips this box.

**If it fails:** `already exists and is not empty`: a real folder is already called `~/BD-Assignment`. Rename it with `mv ~/BD-Assignment ~/BD-Assignment-old` and run the `git clone` line again.

### Step A2 · EVERYONE · Get updates (before every practice and on demo morning)

```
cd ~/BD-Assignment
git pull
```

**Expect:** `Already up to date.` or a list of changed files. Member 1 publishes changes from GitHub Desktop (Commit, then Push).

**If it fails:** `Your local changes ... would be overwritten`: run `git stash`, then `git pull` again.

### Step B · MEMBER 2 (VM2) · Broker 1 config, cluster ID, storage, firewall

```
export KAFKA_HOME=~/kafka
cd ~/BD-Assignment
$KAFKA_HOME/bin/kafka-server-stop.sh
while pgrep -f kafka.Kafka > /dev/null; do sleep 1; done
sed -e "s/VM2_ADDRESS/10.31.88.79/g" -e "s#/var/lib/kafka/data#$HOME/kafka-data#" \
    kafka/vm/vm2-broker1.properties > ~/broker1.properties
grep -E "voters|advertised|log.dirs" ~/broker1.properties
$KAFKA_HOME/bin/kafka-storage.sh random-uuid > ~/cluster.id
cat ~/cluster.id
$KAFKA_HOME/bin/kafka-storage.sh format -t "$(cat ~/cluster.id)" -c ~/broker1.properties
sudo ufw allow in on zt+
```

**Expect:** the `grep` shows `10.31.88.79` (not `VM2_ADDRESS` or `localhost`) and `log.dirs=/home/...`; `cat` prints the cluster ID: **send it to Member 3**; `format` ends with `Formatting ...`. The `ufw` line lets ZeroTier traffic through the firewall. If `format` says the directory is already formatted, run `rm -rf ~/kafka-data` and repeat that line. Box B makes a **new** cluster ID each time, so if you run it again, redo box C too.

### Step C · MEMBER 3 (VM3) · Broker 2 config and storage (on VM3)

```
export KAFKA_HOME=~/kafka
cd ~/BD-Assignment
sed -e "s/VM2_ADDRESS/10.31.88.79/g" -e "s/VM3_ADDRESS/VM3_ZEROTIER_IP/g" \
    -e "s#/var/lib/kafka/data#$HOME/kafka-data#" \
    kafka/vm/vm3-broker2.properties > ~/broker2.properties
grep -E "voters|advertised|log.dirs" ~/broker2.properties
$KAFKA_HOME/bin/kafka-storage.sh format -t CLUSTER_ID_FROM_MEMBER_2 \
    -c ~/broker2.properties
sudo ufw allow in on zt+
```

**Expect:** the `grep` shows VM2's IP in `voters` and VM3's IP in `advertised`. Replace `CLUSTER_ID_FROM_MEMBER_2` with the ID Member 2 sent. Never run `random-uuid` on VM3.

## Before the checkpoints

## Step 0 · EVERYONE · Set up the terminal (every terminal you use)

**Member 1 (VM1):**

```
cd /mnt/c/Users/Aryan/Desktop/BD-Assignment
source .venv-wsl/bin/activate
export KAFKA_HOME=~/kafka
export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64
unset SPARK_HOME HADOOP_HOME HADOOP_CONF_DIR PYSPARK_PYTHON PYSPARK_DRIVER_PYTHON
export B1=10.31.88.79:9092
export B2=VM3_ZEROTIER_IP:9092
export BS=$B1
export BOOTSTRAP=$B1,$B2
export KAFKA_BOOTSTRAP_SERVERS=$B1,$B2
```

**Members 2 and 3 (VM2, VM3):**

```
cd ~/BD-Assignment
git pull
export KAFKA_HOME=~/kafka
export B1=10.31.88.79:9092
export B2=VM3_ZEROTIER_IP:9092
export BS=$B1
export BOOTSTRAP=$B1,$B2
export KAFKA_BOOTSTRAP_SERVERS=$B1,$B2
```

**Member 4 (VM4):**

```
cd ~/BD-Assignment
git pull
source .venv/bin/activate
export KAFKA_HOME=~/kafka
export B1=10.31.88.79:9092
export B2=VM3_ZEROTIER_IP:9092
export BS=$B1
export BOOTSTRAP=$B1,$B2
export KAFKA_BOOTSTRAP_SERVERS=$B1,$B2
```

**Practising alone** (one laptop, local cluster):

```
cd /mnt/c/Users/Aryan/Desktop/BD-Assignment
source .venv-wsl/bin/activate
export KAFKA_HOME=~/kafka
export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64
unset SPARK_HOME HADOOP_HOME HADOOP_CONF_DIR PYSPARK_PYTHON PYSPARK_DRIVER_PYTHON
export B1=localhost:9092
export B2=localhost:9094
export BS=$B1
export BOOTSTRAP=$B1,$B2
export KAFKA_BOOTSTRAP_SERVERS=$B1,$B2
```

**Expect:** the prompt starts with `(.venv-wsl)` (Member 1) or `(.venv)` (Member 4).

**If it fails:** `No such file or directory` on the `cd` line: the repo is in another folder, so do one-time box A. `python: command not found` later: this box wasn't run in that terminal, so run it again. A > prompt where nothing happens: part of a line (often a quote) was missed when copying, so press Ctrl+C and copy the whole box again.

### Step 1 · MEMBER 2 (VM2) · Start broker 1 and the controller

**Demo day:**

```
$KAFKA_HOME/bin/kafka-server-start.sh -daemon ~/broker1.properties
```

**Practising alone** (one laptop, local cluster):

```
bash kafka/local/cluster.sh format
bash kafka/local/cluster.sh start 1
```

**Expect:** no output. (Alone: `format` is needed after every WSL restart and is safe to repeat.)

**If it fails:** if the broker doesn't come up and `$KAFKA_HOME/logs/kafkaServer.out` says `class file versions up to 55.0`, Java is too old: run `export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64` and start it again.

### Step 2 · MEMBER 3 (VM3) · Start broker 2

**Demo day:**

```
$KAFKA_HOME/bin/kafka-server-start.sh -daemon ~/broker2.properties
```

**Practising alone** (one laptop, local cluster):

```
bash kafka/local/cluster.sh start 2
```

**Expect:** no output.

### Step 3 · MEMBER 2 (VM2) · Create fresh topics

```
bash kafka/topics.sh delete
sleep 5
bash kafka/topics.sh create
```

**Expect:** both topics with `PartitionCount: 2`, `ReplicationFactor: 2`, Isr 1,2.

**If it fails:** `Permission denied`: type it with `bash` in front, as here (or run `chmod +x kafka/*.sh kafka/local/*.sh` once). `Connection to node ... could not be established`: a broker isn't running, so redo steps 1 and 2.

### Step 4 · MEMBERS 1 + 4 (VM1 / VM4) · Clear old outputs

```
rm -rf outputs/*.jsonl outputs/*.csv outputs/failover_test outputs/recovery_test
mkdir -p outputs/failover_test outputs/recovery_test
```

**Expect:** no output. The TA only accepts outputs made during the live run.

## Checkpoint 1: cluster

### Step 5 · MEMBER 2 (VM2) · Brokers reachable, VM2 is the controller, topics exist

```
$KAFKA_HOME/bin/kafka-broker-api-versions.sh --bootstrap-server "$BS" | grep "id:"
$KAFKA_HOME/bin/kafka-metadata-quorum.sh --bootstrap-server "$BS" describe --status
$KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server "$BS" --list
```

**Expect:** two brokers, `id: 1` and `id: 2`; `LeaderId: 1` (VM2 is the controller); `orders` and `order_metrics`.

**If it fails:** the brokers are shown as `localhost`: the configs from one-time boxes B and C aren't being used, so redo them, then steps 1 and 2. `Connection to node -1 ... could not be established` or `Connection refused`: broker 1 isn't running, so run step 1, wait 10 s, then step 2. `timed out`: the broker machine's firewall is blocking, so run `sudo ufw allow in on zt+` there.

## Checkpoint 2: raw orders, VM1 -> Kafka -> VM4

### Step 6 · MEMBER 2 (VM2) · Observer on orders (leave it running)

```
$KAFKA_HOME/bin/kafka-console-consumer.sh --bootstrap-server "$BS" \
    --topic orders --group ta-observer-teamXX-run1 \
    --consumer-property auto.offset.reset=latest \
    --max-messages 3 --timeout-ms 30000
```

**Expect:** it waits, and gives up after 30 s, so step 7 must start right away. The `deprecated` warning is harmless.

**If it fails:** it ends with `Processed a total of 0 messages`: step 7 started too late. Run this again, then step 7 straight away.

### Step 7 · MEMBER 1 (VM1) · Producer 1

```
python producer/producer1.py --input order.jsonl
```

**Expect:** `published 62 records`; the observer prints 3 records and `Processed a total of 3 messages`.

**If it fails:** `python: command not found`: run your step 0 box in this terminal. `NoBrokersAvailable` or `KafkaTimeoutError`: this laptop can't use the brokers; step 5 must show `10.31.88.79` for both brokers (not `localhost`), and step 0 must have been run here.

### Step 8 · MEMBER 4 (VM4) · Consumer 1

```
python consumer/consumer1.py --output outputs/orders_landed.jsonl
```

**Expect:** `landed 62 records`.

### Step 9 · MEMBER 4 (VM4) · Show the landed file

```
wc -l outputs/orders_landed.jsonl && head -n 2 outputs/orders_landed.jsonl
```

**Expect:** `62 outputs/orders_landed.jsonl` and two order lines.

## Checkpoint 3: enriched orders, VM4 -> Kafka -> VM1

### Step 10 · MEMBER 4 (VM4) · Spark Job 1

```
python spark/spark_job1.py --input outputs/orders_landed.jsonl \
    --output outputs/order_metrics.jsonl
```

**Expect:** `enriched records written: 44`.

**If it fails:** `'JavaPackage' object is not callable`: an old Spark install is being picked up; run the `unset ...` line from Member 1's step 0 box, then run this again.

### Step 11 · MEMBER 2 (VM2) · Observer on order_metrics (leave it running)

```
$KAFKA_HOME/bin/kafka-console-consumer.sh --bootstrap-server "$BS" \
    --topic order_metrics --group ta-observer-teamXX-run2 \
    --consumer-property auto.offset.reset=latest \
    --max-messages 3 --timeout-ms 30000
```

**Expect:** it waits; step 12 must start within 30 s.

**If it fails:** it ends with `Processed a total of 0 messages`: step 12 started too late. Run this again, then step 12 straight away.

### Step 12 · MEMBER 4 (VM4) · Producer 2

```
python producer/producer2.py --input outputs/order_metrics.jsonl
```

**Expect:** `published 44 records`; the observer prints 3 records.

### Step 13 · MEMBER 1 (VM1) · Consumer 2

```
python consumer/consumer2.py --output outputs/order_metrics_landed.jsonl
```

**Expect:** `landed 44 records`.

### Step 14 · MEMBER 1 (VM1) · Show the landed file

```
wc -l outputs/order_metrics_landed.jsonl && head -n 2 outputs/order_metrics_landed.jsonl
```

**Expect:** `44 outputs/order_metrics_landed.jsonl` and two enriched lines.

### Step 15 · MEMBER 1 (VM1) · Spark Job 2 (the final CSVs)

```
python spark/spark_job2.py --input outputs/order_metrics_landed.jsonl \
    --output-dir outputs
```

**Expect:** `6 rows -> restaurant_summary.csv` and `4 rows -> city_summary.csv`. If the TA wants to see them: `cat outputs/restaurant_summary.csv outputs/city_summary.csv`.

**If it fails:** `'JavaPackage' object is not callable`: an old Spark install is being picked up; run the `unset ...` line from Member 1's step 0 box, then run this again.

## Checkpoint 4: broker failure and recovery

### Step 16 · MEMBER 2 (VM2) · Leaders and ISR before the failure

```
$KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server "$BS" --describe --topic orders
```

**Expect:** `Isr: 1,2` / `2,1` on both partitions.

### Step 17 · MEMBER 1 (VM1) · Slow producer (leave it running)

```
python producer/producer1.py --file order.jsonl --delay 0.5
```

**Expect:** one line per record, `sent 1/62 -> partition ...`, every half second. Keep it on screen: after the kill in step 18 it pauses briefly, then carries on to `sent 62/62` and `published 62 records` with no error. That is the pipeline continuity the TA checks.

### Step 18 · MEMBER 3 (VM3) · Kill the broker (while step 17 is still sending)

**Demo day:**

```
pkill -9 -f kafka.Kafka
```

**Practising alone** (one laptop, local cluster):

```
pkill -9 -f broker2.properties
```

**Expect:** no output. **Never run this on VM2.**

### Step 19 · MEMBER 2 (VM2) · Leaders and ISR after the failure

```
sleep 10
$KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server "$BS" --describe --topic orders
```

**Expect:** `Leader: 1` and `Isr: 1` on both partitions. A `WARN ... Connection to node 2 ... could not be established` line is normal here: broker 2 is the one that's down.

**If it fails:** the describe never finishes and every node fails: both brokers died (the TA's `pkill -9 -f kafka.Kafka` on a machine that runs both). Run steps 1 and 2, then redo from step 16.

### Step 20 · MEMBER 3 (VM3) · Restart the broker in a second terminal (leave it open)

**Demo day:**

```
KAFKA_HEAP_OPTS='-Xmx512m -Xms512m' $KAFKA_HOME/bin/kafka-server-start.sh \
    ~/broker2.properties
```

**Practising alone** (one laptop, local cluster):

```
bash kafka/local/cluster.sh start 2
```

**Expect:** it keeps printing log lines (some `WARN` ones are normal), including `Kafka Server started`, and doesn't return to the prompt. Run it in a **separate terminal** (with step 0 done) and don't close it or press Ctrl+C: that stops broker 2 again.

**If it fails:** it returns to the prompt by itself: broker 2 didn't start; the last lines it printed say why.

### Step 21 · MEMBER 2 (VM2) · Leaders and ISR after the restart

```
sleep 15
$KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server "$BS" --describe --topic orders
```

**Expect:** `Isr: 1,2` again; leaders are still all 1. Don't go on to step 22 until it says so.

**If it fails:** still `Isr: 1`: broker 2 isn't running. Check the step 20 terminal is still open and printing; if not, run step 20 again, wait 15 s, and repeat this step.

### Step 22 · MEMBER 2 (VM2) · Leader election, then check again

```
$KAFKA_HOME/bin/kafka-leader-election.sh --bootstrap-server "$BS" \
    --election-type preferred --all-topic-partitions
$KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server "$BS" --describe --topic orders
```

**Expect:** `Successfully completed leader election`; the leaders are back to how they were in step 16.
**Demo finished.** (Alone: `bash kafka/local/cluster.sh stop all`.)

**If it fails:** many `PreferredLeaderNotAvailableException` lines and `Isr: 1` in the describe: broker 2 isn't back yet (the preferred leader of partition 0 is broker 2). Do step 20 again in its own terminal, wait until step 21 shows `Isr: 1,2`, then run this again. `Connection to node ... could not be established` and it never finishes: broker 1 is down too, so run step 1, wait 10 s, run step 2, then steps 21 and 22 again.

# 4. Member 1 (VM1): Producer 1, Consumer 2, Spark Job 2

Member 1 runs three programs: **producer1.py** at the start, then later **consumer2.py** and **spark_job2.py** at the end. Two shared helper files sit behind them: `publish.py` and `landing.py`.

## 4.1 producer1.py: post the raw orders

Reads `order.jsonl` line by line and sends each line to topic **orders**. All of the real work is in the shared helper `publish.py`.

- **Sends every line as-is, junk included.** The producer only delivers; cleaning is Spark Job 1's job. If both did some cleaning, the rules would be split across two places.
- **No key.** To get a key (order_id) it would have to read the JSON, and some lines are broken. Without a key Kafka spreads messages randomly over the 2 partitions (e.g. 31 / 31, and the split changes each run).
- **acks="all"**: Kafka only says "received" once *every* in-sync copy has the message. This is the safest setting.
- **retries=10**: if something hiccups (e.g. VM3 goes down mid-send), it tries again automatically.
- **linger_ms=10**: waits up to 10 ms to bundle messages together, so fewer network trips.
- It waits for confirmation of every message (`flush()` + `f.get()`), so the printed "62 sent" means 62 *confirmed*, along with which partition each one landed in.

## 4.2 consumer2.py + landing.py: collect the cleaned orders

Reads *everything* from topic **order_metrics** and saves it to `order_metrics_landed.jsonl` (44 lines). Member 4's consumer1.py is the same code with topic **orders**.

- **Reads all partitions itself** (`assign()`, no consumer group). A consumer group remembers where it stopped, so a second run would get nothing. We want the *whole* topic every time.
- **Always starts from the beginning** (`seek_to_beginning`, offset 0).
- **Knows when to stop.** Normal consumers run forever. Ours stops once it has reached the last message of every partition *and* nothing new has arrived for 10 s (`--idle-timeout`).
- **Can start early.** `--wait-timeout` (default 120 s) lets it wait for the producer.
- **Overwrites the file** (`"w"` mode), so running it twice doesn't double the data.
- Prints how many records came from each partition: **16 from partition 0, 28 from partition 1**.

## 4.3 spark_job2.py: make the two reports

Reads `order_metrics_landed.jsonl` and writes **restaurant_summary.csv** (6 rows) and **city_summary.csv** (4 rows).

| Step | What happens | Why |
|---|---|---|
| 1. Read | Read the file with a fixed schema (types given up front) | Types are guaranteed and the file is read only once |
| 2. Dedup | `dropDuplicates` on all 15 columns | Spec: exact duplicates count once. Removes 0 here because Job 1 already did it, but it protects against a producer being run twice |
| 3. Group | groupBy restaurant_id, then groupBy city | The two analytics the spec asks for |
| 4. Compute | count, averages, delayed count (rows where status = DELAYED), delay_rate_pct = 100 x delayed / count | delay_rate is worked out from the exact counts, and rounded only once at the end |
| 5. Round | `F.round(..., 2)` on every calculated number | Spec rule |

| Step | What happens | Why |
|---|---|---|
| 6. Sort + write | Sort the rows, collect them (just 10 rows) and write with Python's csv module | Spark's own writer makes a *folder* of part-files, but the spec wants one file with an exact name |

`.cache()` keeps the deduplicated data in memory because it is used by two reports. Without it Spark would recompute it for each one.

## 4.4 Final outputs (know these by heart)

| restaurant | orders | avg amount | avg prep | avg delivery | delayed | delay % |
|---|---|---|---|---|---|---|
| R01 | 8 | 548.5 | 13.13 | 22.0 | 4 | 50.0 |
| R02 | 8 | 539.25 | 13.38 | 27.0 | 8 | 100.0 |
| R03 | 7 | 664.0 | 13.86 | 32.0 | 4 | 57.14 |
| R04 | 7 | 639.57 | 12.14 | 37.0 | 7 | 100.0 |
| R05 | 7 | 615.14 | 12.0 | 42.0 | 3 | 42.86 |
| R06 | 7 | 712.14 | 13.43 | 47.0 | 7 | 100.0 |

| city | orders | avg amount | avg delivery | delayed | delay % |
|---|---|---|---|---|---|
| Bengaluru (R01+R02) | 16 | 543.88 | 24.5 | 12 | 75.0 |
| Hyderabad (R06) | 7 | 712.14 | 47.0 | 7 | 100.0 |
| Mumbai (R03+R04) | 14 | 651.79 | 34.5 | 11 | 78.57 |
| Pune (R05) | 7 | 615.14 | 42.0 | 3 | 42.86 |

**Easy cross-check:** each restaurant is in exactly one city, so Pune = R05 and Hyderabad = R06. Bengaluru = R01 + R02: 8+8 = 16 orders, 4+8 = 12 delayed, 12/16 = 75%. Order counts add up to 44; delayed orders add up to 33 of 44. Example: R05 delay % = 100 x 3 / 7 = 42.857 -> **42.86**.

**Your steps (section 3):** 0, 4 before; 7; 13, 14, 15; 17.
**Show the evaluator:** 62 published (TA observer sees 3); 44 landed and `wc -l` = 44; 6 + 4 rows, verify OK; the slow producer reaching 62/62 while VM3 is killed.
**Be ready to explain:** why Producer 1 has no key; why the consumer reads every partition from offset 0 and stops by itself; how Job 2 dedupes, groups and rounds; why the producer survives the failure (acks=all, retries, min.insync.replicas=1).

## 5. Member 2 (VM2): Broker 1 + KRaft controller (the boss)

- Runs Kafka with `process.roles=broker,controller`, `node.id=1`.
- **KRaft** = Kafka manages itself, with no separate ZooKeeper. The controller keeps the cluster's "register" (which brokers are alive, which topics exist, who leads each partition).
- Port **9092** = normal Kafka traffic, port **9093** = controller traffic (only VM2 has it).
- Generates the **cluster ID** once; VM3 must use the same ID to join.
- `min.insync.replicas=1`: writes still succeed with only one copy alive (needed while VM3 is off).
- `auto.create.topics.enable=false`: topics are only created on purpose, so a typo can't create a wrong topic.
- In the demo, Member 2 shows the cluster state: both brokers, the topic config, and the ISR before, during and after the failure (section 8 explains how to read it).

**Your steps (section 3):** 0, 1, 3 before; 5; 6; 11; 16, 19, 21, 22. You run the TA's cluster commands and both observers.
**Show the evaluator:** both broker ids; `LeaderId: 1` (VM2 is the controller); the two topics; the observers printing 3 records each; Isr 1,2 -> Leader 1 / Isr 1 -> Isr 1,2 -> leaders restored by the election.
**Be ready to explain:** KRaft and what the controller does; node.id, listeners vs advertised.listeners, ports 9092/9093; cluster ID; min.insync.replicas=1; Leader vs Replicas vs Isr; preferred leader election; why VM2 can't be stopped.

## 6. Member 3 (VM3): Broker 2 (the backup)

- Runs Kafka with `process.roles=broker` only, `node.id=2`.
- Points at VM2:9093 to join and is formatted with **VM2's cluster ID** (it must not create its own cluster).
- Holds the second copy of every partition. In the failure test, **this is the machine that gets stopped**.
- When it stops, VM2's controller notices the missing heartbeats, removes it from the ISR and makes broker 1 the leader everywhere. When it restarts, it copies what it missed and rejoins the ISR.

**Your steps (section 3):** 0, 2 before; 18, 20. Your broker is the one that fails.
**Show the evaluator:** the broker being killed with `pkill -9` while the slow producer runs, then restarted in the foreground. Then point to Member 2's describe output, where Isr shrinks to 1 and grows back to 1,2.
**Be ready to explain:** what happens step by step when VM3 stops (heartbeats stop, it's fenced, leadership moves to broker 1); how it catches up and rejoins the ISR; why VM3 uses VM2's cluster ID; what RF 2 protects against.

# 7. Member 4 (VM4): Consumer 1, Spark Job 1, Producer 2 (the cleaning step)

- **consumer1.py**: same code as consumer2, but reads topic **orders** -> `orders_landed.jsonl` (62 lines).
- **spark_job1.py**: the heart of the project. It **throws away bad records**, **removes duplicates**, and **adds 7 new columns**. Output: `order_metrics.jsonl` (44 lines).
- **producer2.py**: posts those 44 lines to topic **order_metrics**, **keyed by order_id**. The same key always goes to the same partition, so the split is always **16 / 28**.

**Cleaning funnel: 62 -> 48 -> 44**

| Thrown out because... | Count |
|---|---|
| status is not DELIVERED (T04X0001-6: cancelled, pending, failed, refunded) | 6 |
| no order_id (2 "invalid_order_record" lines) | 2 |
| missing fields ({"order_id":"BROKEN"} and a T040002 copy without a city) | 2 |
| bad timestamp (T040005 "bad", T040003 "not-a-timestamp") | 2 |
| distance_km <= 0 (T040001: -2) and order_amount <= 0 (T040004: 0) | 2 |
| **Total dropped -> 48 valid** | **14** |
| Exact duplicates removed (T040006, T040017, T040028, T040039) -> **44** | 4 |

**New columns Spark Job 1 adds** (all in minutes, rounded to 2 decimals):

| Column | Formula |
|---|---|
| preparation_time_min | picked_up - accepted |
| delivery_time_min | delivered - picked_up |
| fulfillment_time_min | delivered - order_timestamp |
| delivery_speed_kmph | distance_km / (delivery_time_min / 60) |
| delay_min | max(0, delivered - expected) |
| delivery_status | ON_TIME if delay = 0, else DELAYED |
| delay_category | 0 ON_TIME \| up to 15 SLIGHT \| up to 30 MODERATE \| over 30 SEVERE |

**Midnight trick:** it uses full date+time, not just the time of day. T040023 picked up 23:20 and delivered 00:02 the next day = **42 min**, not -1398. The time zone is set to UTC so daylight saving can't shift anything.

**Your steps (section 3):** 0, 4 before; 8, 9; 10, 12.
**Show the evaluator:** 62 landed and `wc -l` = 62; the drop report (14 dropped, 48 valid, 4 duplicates, 44 written); 44 published, split 16 / 28 (TA observer sees 3).
**Be ready to explain:** each cleaning rule and the 62 -> 48 -> 44 funnel; the 7 new columns and the midnight case (T040023 = 42 min); why Producer 2 is keyed by order_id.

# 8. The failure test explained (whole team)

```
Before (both up):    orders P0  Leader: 2  Replicas: 2,1  Isr: 2,1
                     orders P1  Leader: 1  Replicas: 1,2  Isr: 1,2
After stopping VM3:  orders P0  Leader: 1  Replicas: 2,1  Isr: 1     <- leader moved to 1
                     orders P1  Leader: 1  Replicas: 1,2  Isr: 1
Restart VM3 -> Isr back to 2,1 / 1,2.
```

**Leader** = the broker currently serving that partition. **Replicas** = where copies are *supposed* to live (doesn't change). **Isr** = copies that are currently up to date (shrinks to just 1 while VM3 is off).

Producing still works with VM3 down because the producer uses `acks=all` (wait for every *in-sync* copy, now just broker 1) and the brokers allow `min.insync.replicas=1`. While the leader moves (about 10 s after the kill) sends to that partition fail briefly; the producer retries every second for up to 30 tries, so no record is lost.

**Preferred leader election** (step 22): each partition's *preferred* leader is the first broker in its `Replicas` list. After VM3 comes back, broker 1 is still leading everything, so the load is uneven. `kafka-leader-election.sh --election-type preferred` moves leadership back to the preferred brokers straight away (Kafka would also do it by itself after a few minutes).

**kill -9 vs a normal stop:** `pkill -9` kills the broker instantly, like a crash or power cut, so it can't hand over its partitions politely. The controller only notices when the broker's heartbeats stop.

# 9. Key words in one line each

| Term | Simple meaning |
|---|---|
| Broker | A Kafka server that stores messages (VM2, VM3). |
| Controller | The boss broker that tracks who is alive and picks leaders (only VM2). |
| KRaft | Kafka's built-in way of electing and storing the boss's records. It replaced ZooKeeper. |
| Topic | A named mailbox: *orders*, *order_metrics*. |
| Partition | A topic is split into pieces (we use 2) so work can be shared. Order is kept only inside one partition. |
| Offset | A message's position number inside a partition (0, 1, 2 ...). |
| Replication factor 2 | Every partition is stored twice, once on each broker. |
| Leader / follower | The leader copy handles reads and writes; the follower copies it. |
| ISR | In-Sync Replicas: the copies that are fully up to date. |
| acks=all | The producer gets "OK" only when all in-sync copies have the message. |
| Bootstrap servers | The first broker address(es) a client contacts to discover the whole cluster. |
| advertised.listeners | The address a broker tells clients to use. It must be the ZeroTier IP, not localhost. |
| ZeroTier | A virtual LAN, so the 4 laptops on different home networks can reach each other. |
| local[*] | Spark runs on this one machine, using all CPU cores. |
| Lazy evaluation | Spark only plans the work; it runs when an action (count, collect) is called. |

# 10. Top viva questions, short answers

### Q. Why use Kafka instead of just copying files?

It separates the stages (VM1 and VM4 never talk directly), stores the data reliably in two copies, and lets any stage re-read the data. The spec also forbids copying the file.

### Q. Why is Producer 1 unkeyed but Producer 2 keyed?

Raw lines may be broken JSON, so Producer 1 doesn't read them to find a key. Cleaned lines are always valid, so Producer 2 keys by order_id, which gives a fixed partition split (16/28).

### Q. Why doesn't the consumer use a consumer group?

Groups remember where they stopped, so a second run would read nothing. Ours is a batch job that must copy the whole topic every time, so it reads all partitions from offset 0 and commits nothing.

### Q. How does the consumer know when to stop?

When its position equals the last offset of every partition and nothing new has arrived for 10 s.

### Q. What happens when VM3 stops?

VM3 stops sending heartbeats, the controller on VM2 removes it from the ISR and makes broker 1 the leader everywhere. Broker 1 already has a full copy (RF 2), so no data is lost.

### Q. Why stop VM3 and not VM2?

VM2 is the only controller. Without it nobody can elect leaders. Real high availability needs 3 controllers.

### Q. Can you still produce while VM3 is down?

Yes. acks=all only waits for the in-sync copies (now just broker 1), and min.insync.replicas=1 allows that.

### Q. What if Producer 1 runs twice?

The topic has 124 messages, but Spark Job 1 still outputs 44 because duplicates are exact and get removed. Delivery is at-least-once, which is why both jobs dedupe.

### Q. Why collect() and write the CSV with Python?

Spark's writer makes a folder of part-files. The spec wants single files with exact names, and the results are tiny (6 and 4 rows).

### Q. Why is dropDuplicates in Job 2 if it removes 0?

The spec says both jobs must dedupe. It also protects against Kafka redelivery or a producer re-run.

### Q. Why not deduplicate by order_id?

The spec says *exact* duplicates. Choosing between two different records with the same ID would be inventing a business rule.

### Q. How do you know the outputs are correct?

tests/verify_outputs.py redoes everything in plain Python (no Spark) and compares every value. It prints "OK: 44 enriched records and both summaries match".

### Q. Why Spark for only 62 records?

The assignment is about the architecture. The same code would work on millions of rows or on a real cluster.

## 11. Numbers to remember

| Item | Value |
|---|---|
| Raw lines | 62 |
| Dropped by cleaning | 14 -> 48 valid |
| Exact duplicates removed | 4 -> 44 enriched |
| orders partition split | random, e.g. 31 / 31 |
| order_metrics partition split | always 16 / 28 |
| Restaurants / cities | 6 (R01-R06) / 4 (Bengaluru, Hyderabad, Mumbai, Pune) |

| Item | Value |
|---|---|
| Delayed orders | 33 of 44 (11 on time) |
| Delay categories | ON_TIME 11, SLIGHT 11, MODERATE 15, SEVERE 7 |
| Highest / lowest delay % | 100% (R02, R04, R06, Hyderabad) / 42.86% (R05, Pune) |
| Fastest / slowest delivery | R01 22.0 min / R06 47.0 min |
| Node IDs / ports | VM2 = node 1 (9092 + 9093); VM3 = node 2 (9092) |
| Versions | Kafka 4.3.1 (KRaft), PySpark 4.2.0, kafka-python 2.2.15, Java 17+ |